Compute, parameters and a constrained hero budget

Start with three separate quantities

A model has stored weights, a token executes selected operations, and a training system consumes wall time. Their relation is conditional. Sparse routing, tied weights, recurrence, quadratic attention, recomputation and communication prevent one parameter count from answering every question.

Use claim `claim-looped-compute-accounting` for the local analytical fixture, and the existing counting guide for broader conventions. The fixture is deliberately small and explicit; it is not the proposed hero architecture.

Derive the matrix count from the declared geometry

Let residual width be h, query heads H_q, KV heads H_{kv}, per-head dimension d, unique blocks L, routed experts per block E, experts selected per token k, expert hidden dimension f, and vocabulary size V. For standard separate Q/K/V/output projections without bias,

$$P_{attn} = hH_qd + 2hH_{kv}d + (H_qd)h.$$

The last matrix maps the concatenated query-head representation back to residual width. We do **not** impose $H_qd = h$: implementations can choose a wider attention projection. The actual configuration and tensor shapes decide the count.

For a SwiGLU expert with gate/up/down matrices,

$$P_{expert} = 3hf, \qquad P_{router} = hE.$$

For untied input embedding and output head, an illustrative matrix-only stored count is

$$P_{stored} = 2Vh + L(P_{attn} + EP_{expert} + P_{router}).$$

A single routing visit through every unique block has core matrix work proportional to

$$P_{core,visit} = L(P_{attn} + kP_{expert} + P_{router}).$$

This last expression is a work proxy. It is not the number of distinct scalar weights touched across a batch, which depends on the routing distribution. Shared experts, dense-only blocks, padding, biases, norms and tied matrices require explicit additions or subtractions. Count the tensors in the qualified trainer to close the accounting.

Weight reuse changes execution, not storage

Suppose the same unique block is visited r times. Its stored tensor is counted once. Its matrix operations are counted on every visit. Routing may select different experts at different visits; “two loops” therefore does not guarantee twice as many **unique** active weights.

Ignoring smaller operations and using three times forward cost as the forward/backward approximation,

$$F_{matrix/token} \approx 6(rP_{core,visit} + Vh).$$

The head term is shown once because the fixture predicts once after the final visit. A recurrent architecture that predicts or applies auxiliary heads at intermediate visits needs additional terms.

For full causal attention implemented with the conventional dense QK/AV work estimate, a useful additional training-work estimate is

$$F_{attention/token} \approx 12rLSH_qd,$$

where S is the packed sequence length. The exact constant depends on which triangular operations are executed and the backward implementation. Sliding windows, grouped-query KV storage, sparse attention and context parallelism affect different terms; do not replace the attention compute term by a KV-head count just because KV storage uses fewer heads.

Reproduce the arithmetic

```
python scripts/kb_workbench.py compute-ledger
```

The receipt specifies $h = 1024, L = 16, E = 32, k = 2, f = 1024, H_q = 16, H_{kv} = 4, d = 64, V = 65536, r = 2, S = 4096$. It yields1,787,297,792 stored matrix parameters and approximately3,730,833,408 training FLOPs/token under the stated approximation. These values exclude the declared smaller operations. They are not measured hardware cost or evidence that this shape is optimal.

Two independent checks are needed before using such a counter to price training: enumerate the trainer's stored tensors with sharing/padding resolved; profile representative steps with the intended sequence length, expert distribution, precision and parallelism. Record whether useful model FLOPs or physical executed FLOPs define the reported rate.

Translate a GPU budget without double counting

For G GPUs, t hero days, productive-wall fraction u and assumed useful model FLOP rate R per GPU,

$$C_{available} = G\,t\,86400\,uR, \qquad D_{compute} = C_{available}/F_{token}.$$

The current planning claim is96 GPUs×30total days=2880GPU-days, with576prehero and2304hero GPU-days. If the30days instead mean hero days plus separately funded preparation, use a different ledger. Do not charge the same GPU-days twice or assume data preparation uses no GPU resources.

The fixture's250TFLOP/s/GPU and90% productive fraction are assumptions. If R is already measured over end-to-end wall time including downtime, applying u again discounts twice. If R is useful training FLOPs divided by training step time, use u only for time outside productive training. Report the denominator explicitly.

Add the constraints compute cannot express

Data supply limits usable exposure. Serving memory depends on **stored** weights, KV cache, concurrency and execution overhead. A BF16 weight array costs roughly $2P_{stored}$bytes before layout/padding. Training state depends on weight, gradient, master-weight and moment dtypes, and on how each is sharded. BF16weights+FP32gradients+FP32master+twoFP32moments is18bytes/parameter before activation, routing, temporary buffers and fragmentation; BF16gradients change that subtotal. Muon-style workspaces need their own accounting.

The decision is therefore constrained optimization over quality, data exposure, training wall time and serving cost. A FLOP-feasible model can still be infeasible to serve, impossible to initialize under the desired mesh, or unsupported by accessible data. The next step is the mixture/data contract, followed by an actual trainer rehearsal.